

Systems To Practice

1. Overview

System design practice means learning how to design real software systems at scale. Instead of only writing functions or solving algorithm problems, you explain how multiple services, databases, queues, caches, APIs, and users interact.

For placements and SDE interviews, common practice systems include:

- URL shortener
- Chat app
- Notification system
- File storage system
- News feed
- Rate limiter
- Search autocomplete
- Video upload system
- Job queue
- E-commerce order system

Definition

A system design problem asks you to design a working architecture for a product or backend service. You usually discuss requirements, APIs, data model, high-level architecture, scaling, reliability, consistency, and trade-offs.

Why It Matters

Real software must handle more than correct logic. It must handle:

- Many users
- Large data
- Failures
- Latency
- Security
- Cost
- Future growth

Where It Is Used In Real Systems

System	Real-World Examples
URL shortener	bit.ly, TinyURL, short links in marketing tools
Chat app	WhatsApp, Slack, Discord, Teams
Notification system	Email, SMS, push notifications, OTP systems

System	Real-World Examples
File storage	Google Drive, Dropbox, S3-like storage
News feed	Instagram, LinkedIn, Twitter/X, Facebook
Rate limiter	API gateways, login protection, payment APIs
Search autocomplete	Google search, Amazon search, IDE suggestions
Video upload	YouTube, Instagram Reels, learning platforms
Job queue	Background emails, image processing, billing jobs
E-commerce order	Amazon, Flipkart, Shopify, food delivery checkout

Why Interviewers Ask About It

Interviewers want to check whether you can:

- Clarify requirements before designing
- Break a large system into smaller components
- Choose suitable databases and storage
- Handle scale, failures, and edge cases
- Explain trade-offs clearly
- Think like an engineer building a real product

2. Core Idea

The core idea is to convert a vague product requirement into a reliable technical architecture.

Intuition

A system is like a city. Roads are APIs, buildings are services, warehouses are databases, traffic signals are rate limiters, delivery trucks are queues, and emergency plans are failure handling.

Good design is not about using every fancy technology. It is about choosing the simplest architecture that satisfies the requirements.

Real-World Analogy

Imagine designing a restaurant:

- Customers place orders: client requests
- Waiters take orders: API servers
- Kitchen prepares food: backend services
- Pantry stores ingredients: database/storage
- Queue manages order sequence: message queue
- Manager prevents overcrowding: rate limiter
- Receipts record payments: transaction log

If only 10 customers come daily, one chef is enough. If 10 lakh customers come daily, you need multiple counters, kitchens, queues, inventory systems, delivery tracking, and failure recovery.

Small Example: URL Shortener

Input:

```
https://example.com/very/long/path/to/article
```

Output:

```
https://short.ly/aB91xZ
```

When a user opens the short URL:

```
short.ly/aB91xZ -> lookup aB91xZ in database -> redirect to long URL
```

Step-By-Step System Design Approach

1. Clarify requirements.
2. Estimate scale.
3. Define APIs.
4. Design the data model.
5. Draw high-level architecture.
6. Discuss storage, cache, queues, and services.
7. Handle failures and edge cases.
8. Discuss trade-offs and improvements.

3. Important Subtopics

3.1 Requirement Clarification

Requirement clarification means asking what the system must and must not do.

It matters because system design depends heavily on constraints. A chat app for 100 users is very different from WhatsApp.

Example questions:

- Should chat support one-to-one messages or groups?
- Should file storage support sharing?
- Should notifications be real-time or eventually delivered?
- Should URL short links expire?

Common interview angle: Interviewers expect you to ask clarifying questions before jumping to architecture.

3.2 Functional vs Non-Functional Requirements

Functional requirements describe features. Non-functional requirements describe quality attributes.

Type	Meaning	Example
Functional	What the system does	Upload file, send message, create order
Non-functional	How well it does it	Low latency, high availability, consistency

Why it matters: Most design decisions come from non-functional requirements.

Example: A payment system needs strong consistency. A news feed can tolerate eventual consistency.

Common interview angle: Interviewers check whether you separate features from scale and reliability needs.

3.3 API Design

API design defines how clients interact with the system.

Example for URL shortener:

```
POST /shorten
GET /{shortCode}
```

Why it matters: APIs reveal the system's contract.

Common interview angle: Interviewers expect clean endpoints, request/response examples, and error cases.

3.4 Data Modeling

Data modeling means deciding what entities to store and how they relate.

Example for e-commerce order system:

Entity	Important Fields
User	user_id, name, email
Product	product_id, price, stock
Cart	cart_id, user_id
Order	order_id, user_id, status, total_amount
Payment	payment_id, order_id, status

Why it matters: A poor data model makes scaling and correctness difficult.

Common interview angle: Interviewers may ask about SQL vs NoSQL, indexes, primary keys, and consistency.

3.5 Caching

Caching stores frequently accessed data in fast memory.

Example:

- Cache short URL mappings.
- Cache user profiles in chat.
- Cache popular feed posts.
- Cache autocomplete prefixes.

Why it matters: Caching reduces latency and database load.

Common interview angle: Cache invalidation, TTL, cache miss, cache stampede.

3.6 Load Balancing

A load balancer distributes traffic across multiple servers.

Example:

```
Users -> Load Balancer -> API Server 1
                        -> API Server 2
                        -> API Server 3
```

Why it matters: It improves availability and handles more traffic.

Common interview angle: What happens when one server fails?

3.7 Database Choice

Choosing SQL or NoSQL depends on query patterns, consistency, and scale.

Use Case	Common Choice	Reason
Orders/payments	SQL	Transactions and consistency
Chat messages	NoSQL or wide-column DB	High write volume
URL mappings	Key-value store	Fast lookup
Search autocomplete	Trie, search engine, key-value store	Prefix lookup
File metadata	SQL/NoSQL	Depends on sharing and query needs

Common interview angle: Do not say "NoSQL is always scalable" or "SQL cannot scale." Explain trade-offs.

3.8 Queues And Background Processing

A queue stores tasks to be processed asynchronously.

Example:

```
User uploads video -> Queue -> Transcoding workers -> Storage -> Notification
```

Why it matters: Queues improve reliability and prevent slow work from blocking users.

Common interview angle: At-least-once delivery, retries, dead-letter queue, idempotency.

3.9 Consistency And Availability

Consistency means users see correct/latest data. Availability means the system responds even during failures.

Example:

- Order payment requires strong consistency.
- News feed ranking can be eventually consistent.
- Notification delivery can retry later.

Common interview angle: Interviewers expect trade-off thinking, not memorized CAP theorem lines.

3.10 Observability

Observability means knowing what is happening inside the system using logs, metrics, and traces.

Example metrics:

- Request latency
- Error rate
- Queue depth
- Notification delivery success
- Cache hit rate
- Payment failure rate

Common interview angle: How will you debug production issues?

4. Real-World Example

Example: Video Upload System

A video upload platform like YouTube needs many backend components.

```
Client
|
v
Upload API
|
v
Object Storage <----- Metadata DB
|
v
Message Queue
|
v
Transcoding Workers
|
v
```

```

Processed Video Storage
|
v
CDN
|
v
Viewer

```

How it works:

1. User uploads a video.
2. Upload service stores the raw file in object storage.
3. Metadata is saved in a database.
4. A message is pushed to a queue.
5. Workers transcode video into multiple resolutions.
6. Processed files are stored.
7. CDN serves video close to users.
8. Notification service can inform subscribers.

Used in:

- Backend server: upload APIs and metadata service
- Distributed system: queues and workers
- Database: metadata, user, video status
- Application code: retry logic and progress updates
- Storage system: object storage and CDN

5. Diagrams / Mental Models

General System Design Template

```

Clients
|
v
Load Balancer
|
v
API Servers
|
+--> Cache
|
+--> Database
|
+--> Message Queue --> Workers
|
+--> Object Storage / Search Index / External Services

```

Read-Heavy vs Write-Heavy Systems

System	Read/Write Pattern	Design Focus
URL shortener	Read-heavy	Cache and fast redirect
Chat app	Write-heavy and real-time	WebSockets, message ordering
News feed	Read-heavy	Feed generation and caching
Notification system	Write-heavy bursts	Queues, retries, provider fallback
Search autocomplete	Read-heavy	Prefix index, low latency
E-commerce order	Correctness-heavy	Transactions and consistency

Synchronous vs Asynchronous Flow

Synchronous:

Client -> API -> Service -> DB -> Response

Asynchronous:

Client -> API -> Queue -> Worker -> DB/Storage -> Later Result

Hot Path vs Background Path

Path	Meaning	Example
Hot path	Must happen before user gets response	Create short URL
Background path	Can happen later	Send analytics event

6. Common Interview Questions

Q1. How would you design a URL shortener?

Answer: Design APIs to create and resolve short URLs. Store shortCode to longURL mapping in a database. Use a unique ID generator or base62 encoding. Cache popular mappings. Redirect using HTTP 301 or 302 depending on whether links are permanent.

Key points expected:

- Short code generation
- Database schema
- Redirect flow
- Cache
- Expiry and analytics

Common mistakes:

- Ignoring collisions
- Not discussing cache
- Forgetting link expiry

- Using random strings without collision handling

Q2. How would you design a chat app?

Answer: Use WebSockets for real-time communication. Store messages in a message database. Maintain connection servers for online users. Use queues or pub/sub for message fanout. Track delivery status and handle offline users.

Key points expected:

- WebSocket vs HTTP polling
- Message ordering
- Online/offline delivery
- Group chat fanout
- Persistence

Common mistakes:

- Assuming all users are always online
- Ignoring message ordering
- Forgetting mobile reconnection
- Not storing messages durably

Q3. How would you design a notification system?

Answer: Accept notification requests through an API, store notification jobs, push them into a queue, and let workers send email, SMS, or push notifications through providers. Use retries, rate limits, templates, and dead-letter queues.

Key points expected:

- Queue-based architecture
- Multiple channels
- Retry and failure handling
- User preferences
- Idempotency

Common mistakes:

- Sending notifications synchronously in the request path
- Ignoring duplicate notifications
- No provider fallback
- No unsubscribe or preference handling

Q4. How would you design a file storage system?

Answer: Store file bytes in object storage and metadata in a database. Support upload, download, sharing, permissions, versioning, and large file multipart upload. Use CDN for downloads and checksums for integrity.

Key points expected:

- Object storage
- Metadata database
- Permissions
- Multipart upload
- CDN

Common mistakes:

- Storing large files directly in relational DB
- Ignoring access control
- Not handling upload failures
- Forgetting deduplication or versioning

Q5. How would you design a news feed?

Answer: Store posts, follow relationships, and feed items. For celebrities or high-fanout users, use hybrid fanout. Cache feed pages. Rank posts based on recency, engagement, and relevance.

Key points expected:

- Fanout on write vs fanout on read
- Ranking
- Caching
- Pagination
- Handling celebrity users

Common mistakes:

- Using only one fanout strategy
- Ignoring ranking
- Offset pagination for very large feeds
- Forgetting privacy rules

Q6. How would you design a rate limiter?

Answer: Track request counts per user, IP, API key, or route. Use algorithms like token bucket, leaky bucket, fixed window, or sliding window. Store counters in Redis for low latency. Return HTTP 429 when limit is exceeded.

Key points expected:

- Rate limit key
- Algorithm choice
- Redis/counter storage
- Distributed consistency
- 429 response

Common mistakes:

- Using only in-memory counters on one server
- Ignoring distributed deployments

- Not defining limit scope
- No TTL on counters

Q7. How would you design search autocomplete?

Answer: Build a prefix index using trie, search engine, or key-value store. Store popular queries and rank suggestions by frequency, recency, personalization, and typo tolerance. Serve from memory/cache for low latency.

Key points expected:

- Prefix lookup
- Ranking
- Index building
- Updates
- Low latency

Common mistakes:

- Scanning the full database per keystroke
- Ignoring ranking
- Not handling spelling errors
- Not discussing data freshness

Q8. How would you design a video upload system?

Answer: Use upload service, object storage, metadata DB, message queue, transcoding workers, processed storage, CDN, and status tracking. Large uploads should be resumable and processed asynchronously.

Key points expected:

- Object storage
- Async transcoding
- Multiple resolutions
- CDN
- Upload status

Common mistakes:

- Transcoding synchronously
- Ignoring large file upload failures
- No progress/status tracking
- No retry handling

Q9. How would you design a job queue?

Answer: Producers push jobs to a queue. Workers consume jobs, process them, acknowledge success, retry failures, and move permanently failing jobs to a dead-letter queue. Use visibility timeout and idempotent workers.

Key points expected:

- Producer-consumer model
- Ack/retry
- Dead-letter queue
- Idempotency
- Delayed and scheduled jobs

Common mistakes:

- Assuming exactly-once execution
- No retry limits
- No monitoring of queue depth
- Non-idempotent job handlers

Q10. How would you design an e-commerce order system?

Answer: Use cart, inventory, order, payment, and shipment services. Create orders transactionally, reserve inventory, process payment, update order state, and publish events for shipment and notification. Use idempotency keys for payment and checkout.

Key points expected:

- Order state machine
- Inventory reservation
- Payment consistency
- Idempotency
- Failure recovery

Common mistakes:

- Reducing stock after payment without reservation
- Ignoring duplicate payment requests
- No order status transitions
- Treating distributed transactions as simple local transactions

Q11. How do you choose SQL vs NoSQL in system design?

Answer: Choose SQL when relationships, transactions, and strong consistency are important. Choose NoSQL when access patterns are simple, scale is large, schema is flexible, or high write throughput is needed.

Key points expected:

- Query pattern
- Consistency needs
- Scale
- Transactions
- Schema flexibility

Common mistakes:

- Saying SQL is not scalable
- Choosing NoSQL without explaining access patterns

- Ignoring indexes

Q12. Why are queues important in system design?

Answer: Queues decouple producers from consumers. They allow slow work to happen asynchronously, smooth traffic spikes, support retries, and improve reliability.

Key points expected:

- Async processing
- Backpressure
- Retry
- Worker scaling
- Failure isolation

Common mistakes:

- Using queues for everything
- Not handling duplicate jobs
- Ignoring queue monitoring

7. Deep-Dive Questions

Q1. How do you handle hot keys in a URL shortener or cache?

Hot keys happen when one key receives extremely high traffic. Use cache replication, CDN-level redirects, local in-memory cache, request coalescing, and sharding strategies. For very popular links, store mappings closer to edge servers.

Q2. How do you guarantee message ordering in a chat app?

Use per-conversation sequence numbers generated by a single authority or partition messages by conversation ID. Store messages in order and let clients reconcile missing messages. Global ordering is usually unnecessary; conversation-level ordering is enough.

Q3. How do you prevent duplicate notifications?

Use an idempotency key such as `notification_id` plus `user_id` plus channel. Store send attempts. Workers should check whether a notification was already sent before sending again. Provider callbacks should also be handled idempotently.

Q4. How do you design inventory reservation in e-commerce?

When checkout starts, reserve inventory for a short TTL. If payment succeeds, convert reservation to confirmed stock deduction. If payment fails or expires, release the reservation. This avoids overselling.

Q5. How do you scale news feed generation for users with millions of followers?

Use hybrid fanout. For normal users, fanout on write into followers' feeds. For celebrity users, do not push to every follower immediately; pull their posts during feed read and merge/rank them dynamically.

8. Comparison Tables

Fanout On Write vs Fanout On Read

Feature	Fanout On Write	Fanout On Read
Meaning	Precompute feed when post is created	Build feed when user opens app
Read latency	Low	Higher
Write cost	High	Low
Best for	Normal users with moderate followers	Celebrity/high-fanout users
Used in	News feed, social apps	News feed, timelines

Token Bucket vs Leaky Bucket

Feature	Token Bucket	Leaky Bucket
Allows bursts	Yes	Usually no
Concept	Tokens refill over time	Requests drain at fixed rate
Best for	API rate limiting with burst tolerance	Smooth traffic shaping
Common use	Public APIs	Network traffic control

Polling vs WebSocket

Feature	Polling	WebSocket
Direction	Client repeatedly asks server	Full-duplex connection
Latency	Higher	Lower
Server cost	Can be wasteful	Connection management needed
Best for	Occasional updates	Real-time chat, live dashboards

SQL vs NoSQL For Practice Systems

System	SQL Fit	NoSQL Fit
E-commerce order	Strong fit for transactions	Useful for logs/events
Chat app	Useful for users/groups	Strong fit for messages
URL shortener	Works well	Key-value store also strong
News feed	Useful for users/follows	Strong fit for feed items
File storage	Useful for metadata	Useful for scalable metadata

Synchronous vs Asynchronous Processing

Feature	Synchronous	Asynchronous
User waits	Yes	No or minimal
Complexity	Lower	Higher
Reliability	Direct failure visible	Needs retry and monitoring
Example	Create order response	Send email, transcode video

Strong Consistency vs Eventual Consistency

Feature	Strong Consistency	Eventual Consistency
Meaning	Reads see latest confirmed write	Reads may temporarily see old data
Latency	Often higher	Often lower
Best for	Payments, orders, inventory	Feed, analytics, notifications
Risk	Lower correctness risk	Temporary stale data

9. Common Mistakes

- Starting with technology before requirements.
- Saying "use microservices" without explaining why.
- Ignoring scale estimates.
- Ignoring database indexes.
- Forgetting cache invalidation.
- Not handling retries and duplicate requests.
- Assuming queues give exactly-once processing.
- Ignoring security and permissions.
- Not discussing failure scenarios.
- Designing everything as strongly consistent even when not needed.
- Using only one database for every workload.
- Forgetting monitoring, logs, metrics, and alerts.

10. Edge Cases / Special Cases

URL Shortener

- Short code collision
- Expired links
- Malicious URLs
- Custom aliases
- Very popular links

Chat App

- Offline users
- Duplicate messages after retry
- Message ordering

- User reconnecting from multiple devices
- Group chat fanout

Notification System

- Provider outage
- Duplicate notifications
- User unsubscribed
- Rate limits from email/SMS providers
- Time-zone based delivery

File Storage

- Interrupted upload
- Permission changes
- File version conflicts
- Virus scanning
- Duplicate file content

News Feed

- Celebrity users
- Blocked users
- Deleted posts
- Privacy settings
- Infinite scroll pagination

Rate Limiter

- Distributed counters
- Clock skew
- Burst traffic
- Shared IP addresses
- Different limits for different users

Search Autocomplete

- Typo tolerance
- Abusive queries
- Fresh trending queries
- Personalization
- Low-latency requirement per keystroke

Video Upload

- Huge files
- Failed transcoding
- Copyright scanning
- Multiple resolutions

- Resume upload

Job Queue

- Worker crash
- Poison messages
- Retry storms
- Dead-letter queue
- Idempotent job execution

E-Commerce Order

- Payment succeeds but order update fails
- Duplicate checkout click
- Inventory overselling
- Refunds
- Partial shipment

11. How to Explain in Interview

"I would start by clarifying the functional requirements and scale. Then I would define APIs, data model, and the high-level architecture. For read-heavy systems like URL shortener or autocomplete, I would focus on caching and low-latency lookups. For write-heavy or async systems like chat, notification, video upload, and job queue, I would use queues, workers, retries, and idempotency. For correctness-heavy systems like e-commerce orders, I would focus on transactions, inventory reservation, payment idempotency, and clear order states. After the basic design, I would discuss bottlenecks, failure cases, monitoring, and trade-offs."

12. Quick Revision Notes

Key Definitions

Term	Meaning
Load balancer	Distributes traffic across servers
Cache	Fast storage for frequently accessed data
Queue	Buffer for async tasks
CDN	Serves static content near users
Sharding	Splitting data across machines
Replication	Copying data for availability/read scale
Idempotency	Same request repeated has same final effect
TTL	Time after which data expires
Backpressure	Slowing producers when consumers cannot keep up

Important Points

- URL shortener is mostly read-heavy.
- Chat needs real-time delivery and message ordering.
- Notification systems need retries and provider fallback.
- File storage separates metadata from file bytes.
- News feed uses fanout strategies.
- Rate limiter usually uses Redis or distributed counters.
- Autocomplete needs prefix search and ranking.
- Video upload needs asynchronous transcoding.
- Job queues need ack, retry, and dead-letter queue.
- E-commerce orders need strong consistency for payments and inventory.

Common Comparisons

Comparison	Remember
Cache vs DB	Cache is fast but temporary; DB is durable
SQL vs NoSQL	SQL for transactions; NoSQL for flexible/high-scale access patterns
Polling vs WebSocket	Polling is simple; WebSocket is real-time
Sync vs Async	Sync blocks user; async uses queue/workers
Strong vs Eventual consistency	Correct now vs correct eventually

Must-Remember Facts

- Always clarify requirements first.
- Mention scale assumptions.
- Design APIs before deep internals.
- Discuss data model and indexes.
- Add cache only where it solves real latency/load problems.
- Queues require idempotent workers.
- Payment and order systems need idempotency keys.
- Monitoring is part of production design.

Interview Traps

- "Exactly once" is usually not guaranteed by queues.
- Cache can return stale data.
- Random short codes can collide.
- Distributed rate limiting is harder than local rate limiting.
- Offset pagination can be slow for large feeds.
- Payment success and order success can diverge.

13. Practice Tasks

Task 1: Design URL Shortener

Draw:

```
Client -> API -> Cache -> DB
```

Write APIs:

```
POST /shorten  
GET /{code}
```

Practice explaining:

- Code generation
- Collision handling
- Redirect status code
- Cache strategy
- Expiry

Task 2: Design Chat App Message Flow

Draw:

```
Sender -> WebSocket Server -> Message Service -> DB  
      |  
      v  
Receiver Connection
```

Practice:

- One-to-one chat
- Group chat
- Offline delivery
- Read receipts
- Message ordering

Task 3: Design Notification Queue

Draw:

```
API -> Notification DB -> Queue -> Workers -> Email/SMS/Push Provider
```

Practice:

- Retry policy
- Dead-letter queue
- User preferences

- Duplicate prevention

Task 4: Design File Upload

Explain:

- Metadata DB
- Object storage
- Multipart upload
- CDN
- Permissions

Task 5: Compare News Feed Strategies

Create a table comparing:

- Fanout on write
- Fanout on read
- Hybrid fanout

Mention which one works for celebrity users.

Task 6: Implement Simple Rate Limiter In Python

```
import time

class FixedWindowRateLimiter:
    def __init__(self, limit, window_seconds):
        self.limit = limit
        self.window_seconds = window_seconds
        self.store = {}

    def allow(self, user_id):
        now = int(time.time())
        window = now // self.window_seconds
        key = (user_id, window)

        count = self.store.get(key, 0)
        if count >= self.limit:
            return False

        self.store[key] = count + 1
        return True

limiter = FixedWindowRateLimiter(limit=3, window_seconds=60)

print(limiter.allow("u1"))
print(limiter.allow("u1"))
print(limiter.allow("u1"))
print(limiter.allow("u1"))
```

Practice explaining why this is not enough for distributed systems.

Task 7: Build Autocomplete Mental Model

For words:

```
car, cat, cart, camera, dog
```

Trace suggestions for:

```
ca -> car, cat, cart, camera  
car -> car, cart
```

Discuss ranking by popularity.

Task 8: Design Video Upload Pipeline

Draw:

```
Upload -> Raw Storage -> Queue -> Transcoder -> Processed Storage -> CDN
```

Practice:

- Large upload handling
- Transcoding failure
- Multiple resolutions
- Status updates

Task 9: Design Job Queue

Explain:

- Producer
- Queue
- Worker
- Ack
- Retry
- Dead-letter queue
- Visibility timeout

Task 10: Design E-Commerce Checkout

Draw:

Cart -> Order Service -> Inventory Reservation -> Payment -> Confirm Order

Practice:

- Duplicate checkout
- Payment failure
- Inventory release
- Order status machine

14. Final Cheat Sheet

Core Definition

System design is the process of designing scalable, reliable, and maintainable software systems using APIs, databases, caches, queues, storage, and services.

Why It Matters

It shows whether you can build real products, not just solve coding problems.

Most Asked Systems

System	Main Focus
URL shortener	Key generation, redirect, cache
Chat app	WebSocket, message ordering, offline delivery
Notification system	Queue, retry, providers
File storage	Object storage, metadata, permissions
News feed	Fanout, ranking, cache
Rate limiter	Token bucket, Redis, distributed counters
Search autocomplete	Prefix index, ranking, low latency
Video upload	Upload, queue, transcoding, CDN
Job queue	Ack, retry, DLQ, idempotency
E-commerce order	Inventory, payment, transactions, order state

Common Comparisons

Compare	Interview Answer
SQL vs NoSQL	SQL for transactions; NoSQL for scale/flexible access patterns
Polling vs WebSocket	Polling is simple; WebSocket is real-time
Cache vs DB	Cache is fast; DB is durable

Compare	Interview Answer
Sync vs Async	Sync waits; async queues work
Strong vs Eventual consistency	Latest correct data vs temporary staleness
Fanout on write vs read	Precompute feed vs build feed at read time

Most Asked Questions

- Design a URL shortener.
- Design WhatsApp-like chat.
- Design a notification service.
- Design Google Drive-like file storage.
- Design Instagram/Twitter news feed.
- Design a distributed rate limiter.
- Design search autocomplete.
- Design YouTube video upload.
- Design a job queue.
- Design Amazon checkout/order system.

One-Line Interview Answer

"I clarify requirements and scale, define APIs and data model, design the main services, choose storage/cache/queue based on access patterns, then discuss consistency, failures, monitoring, and trade-offs."